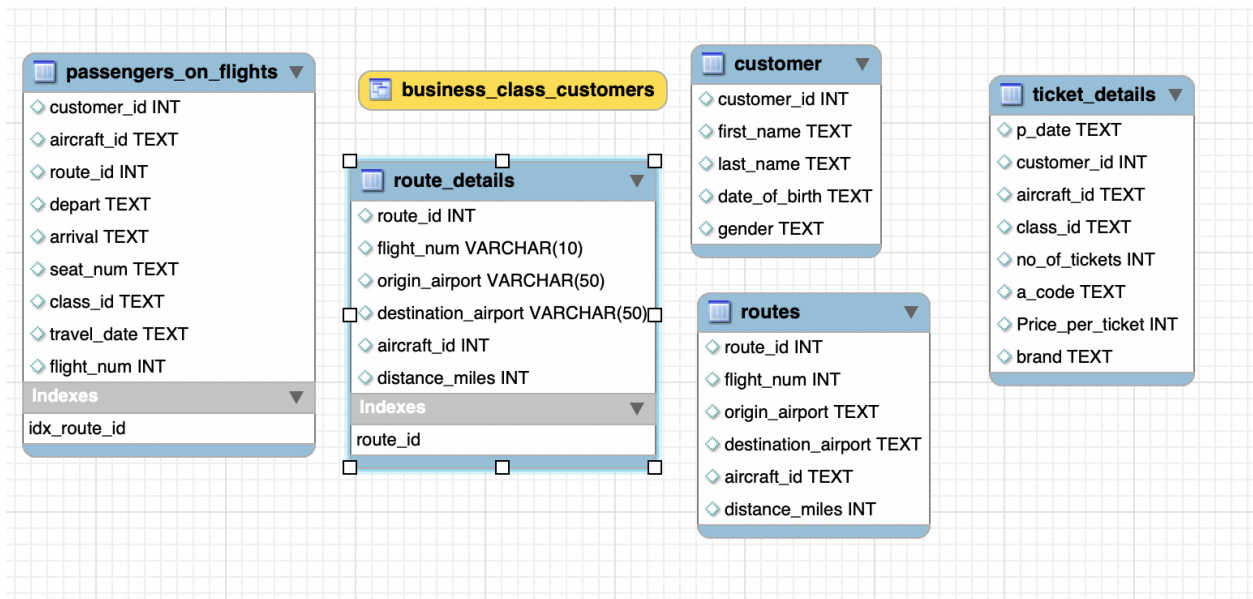


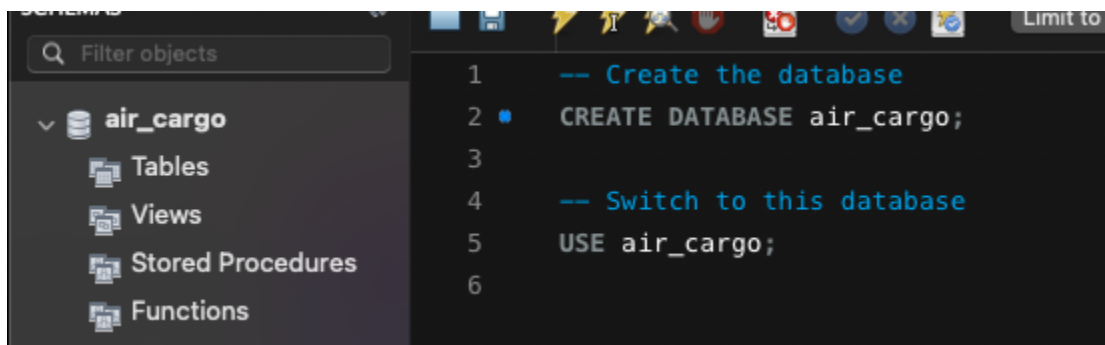
1. ER diagram



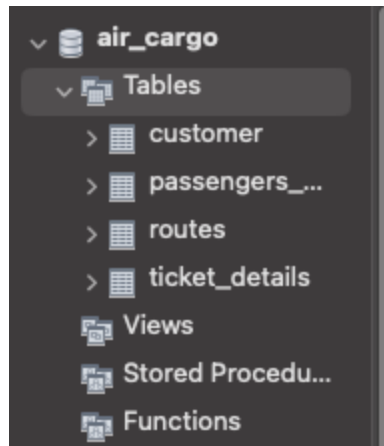
Creating the database for this project first!

```
-- Create the database  
CREATE DATABASE air_cargo;
```

```
-- Switch to this database  
USE air_cargo;
```

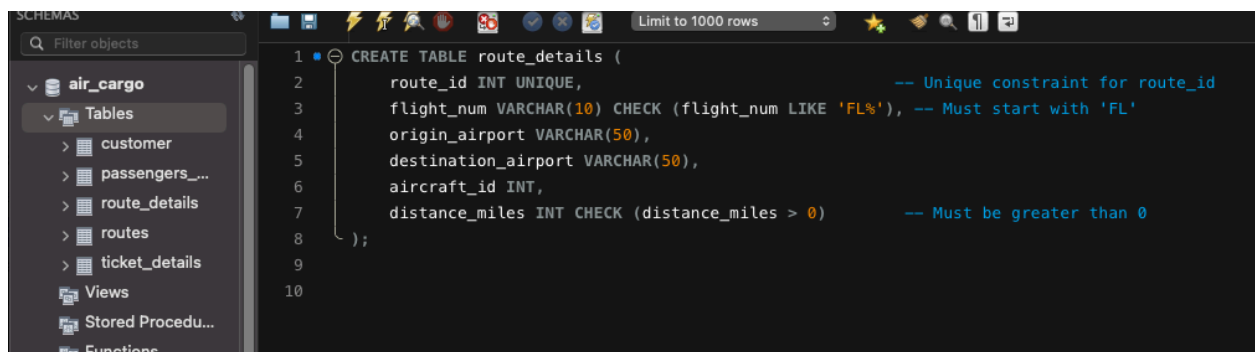


+ Added all datasets to the air_cargo database



2. This query creates the `route_details` table with the respective six fields: `route_id` (unique constraint to prevent duplicates), `flight_num` (uses a `CHECK` constraint to ensure it starts with "FL"), `origin_airport` and `destination_airport` (text fields for airport names), `aircraft_id` (integer for aircraft identification), and `distance_miles` (must be greater than 0 using a `CHECK` constraint). These constraints enforce data accuracy and maintain proper validation rules.

```
CREATE TABLE route_details (
  route_id INT UNIQUE,                -- Unique constraint for route_id
  flight_num VARCHAR(10) CHECK (flight_num LIKE 'FL%'), -- Must start with 'FL'
  origin_airport VARCHAR(50),
  destination_airport VARCHAR(50),
  aircraft_id INT,
  distance_miles INT CHECK (distance_miles > 0)    -- Must be greater than 0
);
```



3. Query displays all the passengers (customers) who have travelled in routes 01 to 25 by taking data from the `passengers_on_flights` table

```
SELECT *
```

```
FROM passengers_on_flights
WHERE route_id BETWEEN 1 AND 25;
```

The screenshot shows a database management tool interface. On the left, a 'SCHEMAS' panel lists various databases including 'air_cargo', 'cyclical_data', 'employees_db', 'information_sche...', 'mysql', 'performance_sch...', 'proj_db', 'sql_hr', 'sql_inventory', 'sql_invoicing', 'sql_store', 'sys', and 'test'. The 'air_cargo' database is selected, showing a list of tables: 'customer', 'passengers_...', 'route_details', 'routes', and 'ticket_details'. The 'passengers_on_flights' table is highlighted.

The main area displays a SQL query:

```
1 SELECT *
2 FROM passengers_on_flights
3 WHERE route_id BETWEEN 1 AND 25;
```

Below the query, the 'Result Grid' shows 23 rows of data. The columns are: customer_id, aircraft_id, route_id, depart, arrival, seat_num, class_id, travel_date, flight_nu... (truncated). The data includes various flight records with different aircraft, routes, and passenger classes.

At the bottom, the 'Action Output' section shows the query execution details: 97 rows returned, 21:16:45 duration, and 0.0018 sec / 0.00003... duration.

4. Query to identify the number of passengers and total revenue in business class from the ticket_details table

Note: Business is spelled incorrectly so was getting null before!

The screenshot shows a database management tool interface. The main area displays a SQL query:

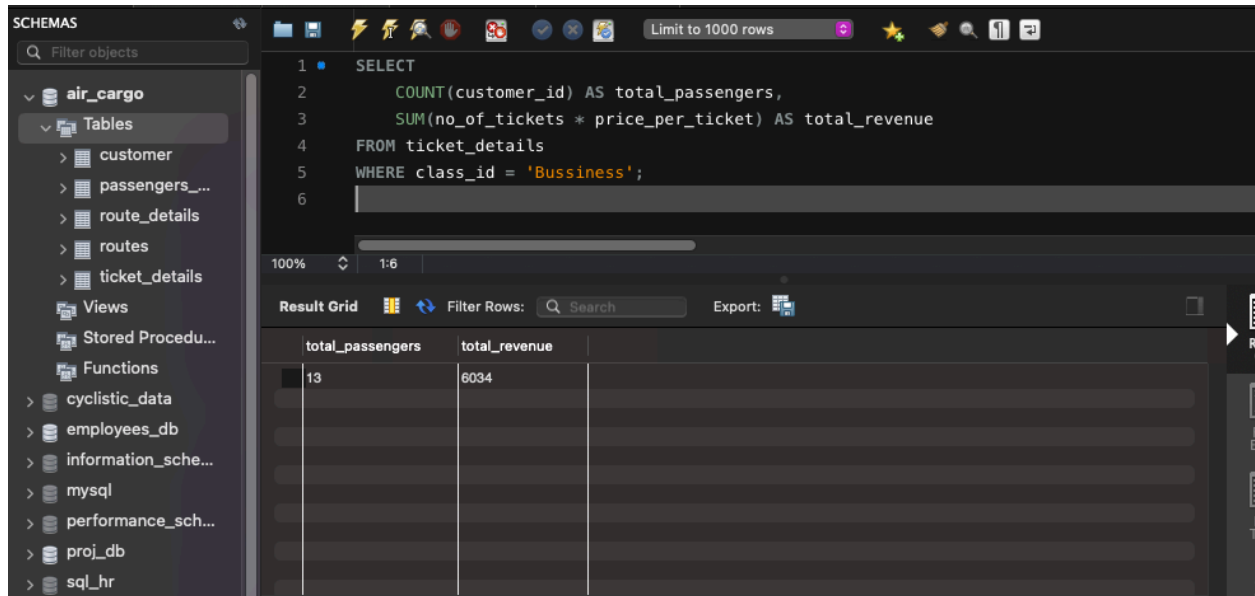
```
1 SELECT DISTINCT class_id
2 FROM ticket_details;
```

Below the query, the 'Result Grid' shows 4 rows of data. The columns are: class_id. The data includes: Economy, Economy Plus, Bussiness, and First Class. Note the spelling 'Bussiness' instead of 'Business'.

Trouble shooted by:

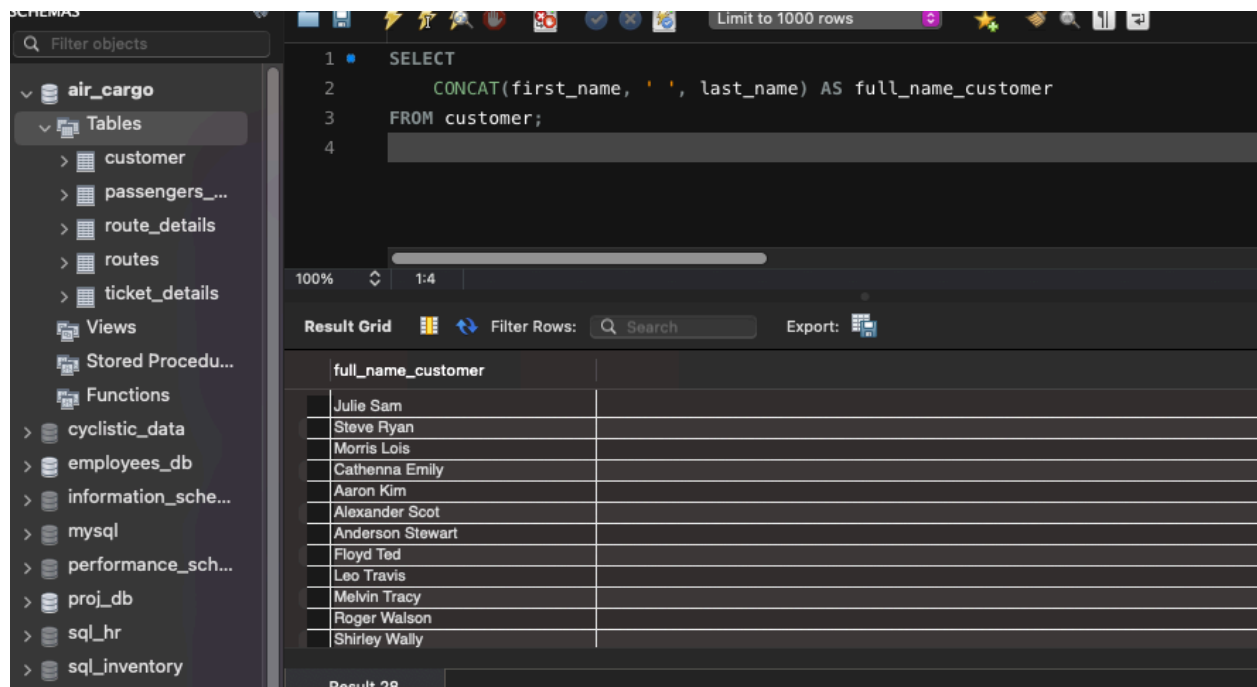
```
SELECT
COUNT(customer_id) AS total_passengers,
```

```
SUM(no_of_tickets * price_per_ticket) AS total_revenue
FROM ticket_details
WHERE class_id = 'Bussiness';
```



5. Query to display the full name of the customer by extracting the first name and last name from the customer table

```
SELECT
    CONCAT(first_name, ' ', last_name) AS full_name_customer
FROM customer;
```



6. Query to extract the customers who have registered and booked a ticket, using data from the customer and ticket_details tables

```
SELECT DISTINCT c.customer_id,  
               CONCAT(c.first_name, ' ', c.last_name) AS full_name_customer  
FROM customer c  
JOIN ticket_details t  
  ON c.customer_id = t.customer_id;
```

The screenshot shows a SQL IDE with a query editor at the top and a result grid at the bottom. The query is as follows:

```

1 SELECT DISTINCT c.customer_id,
2   CONCAT(c.first_name, ' ', c.last_name) AS full_name_customer
3 FROM customer c
4 JOIN ticket_details t
5   ON c.customer_id = t.customer_id;
6

```

The result grid displays the following data:

customer_id	full_name_customer
27	Cherly Vernon
22	Pheny Eri
21	Chirsty Josh
4	Cathenna Emily
5	Aaron Kim
7	Anderson Stewart
8	Floyd Ted
9	Leo Travis
10	Melvin Tracy
11	Roger Walson
19	Joyce Paul
13	Solomon Walter
14	Carol Vernon
25	Moss Morris
16	Chirstine Willis
17	Catherine Shad
18	Gloria Richie
24	Calvin Willis
20	Sara Oliver

7. Query to identify the customer's first name and last name based on their customer ID and brand (Emirates) from the ticket_details table

```

SELECT DISTINCT c.customer_id,
               c.first_name,
               c.last_name
FROM customer c
JOIN ticket_details t
  ON c.customer_id = t.customer_id
WHERE t.brand = 'Emirates';

```

```

1 SELECT DISTINCT c.customer_id,
2     c.first_name,
3     c.last_name
4 FROM customer c
5 JOIN ticket_details t
6     ON c.customer_id = t.customer_id
7 WHERE t.brand = 'Emirates';
8

```

100% 1:8

Result Grid Filter Rows: Search Export:

customer_id	first_name	last_name
2	Steve	Ryan
4	Cathenna	Emily
5	Aaron	Kim
7	Anderson	Stewart
9	Leo	Travis
11	Roger	Walson
14	Carol	Vernon
18	Gloria	Richie
19	Joyce	Paul
25	Moss	Morris
27	Cherly	Vernon
31	James	Robert
44	Billy	Brian
49	Russell	Peter

Result 31 Read Only

8. Query to identify the customers who have travelled by *Economy Plus* class using Group By and Having clause on the passengers_on_flights table.

```

SELECT class_id, COUNT(customer_id) AS total_passengers
FROM passengers_on_flights
GROUP BY class_id
HAVING class_id = 'Economy Plus';

```

```

1 • SELECT class_id, COUNT(customer_id) AS total_passengers
2   FROM passengers_on_flights
3   GROUP BY class_id
4   HAVING class_id = 'Economy Plus';
5
6
7

```

100% 1:5

Result Grid Filter Rows: Search Export:

	class_id	total_passengers
	Economy Plus	10

9. Query to identify whether the revenue has crossed 10000 using the IF clause on the ticket_details table.

```

SELECT
  IF(SUM(no_of_tickets * price_per_ticket) > 10000,
    'Revenue Crossed 10000',
    'Revenue Below 10000') AS revenue_status
FROM ticket_details;

```

```

1 • SELECT
2   IF(SUM(no_of_tickets * price_per_ticket) > 10000,
3     'Revenue Crossed 10000',
4     'Revenue Below 10000') AS revenue_status
5   FROM ticket_details;
6
7

```

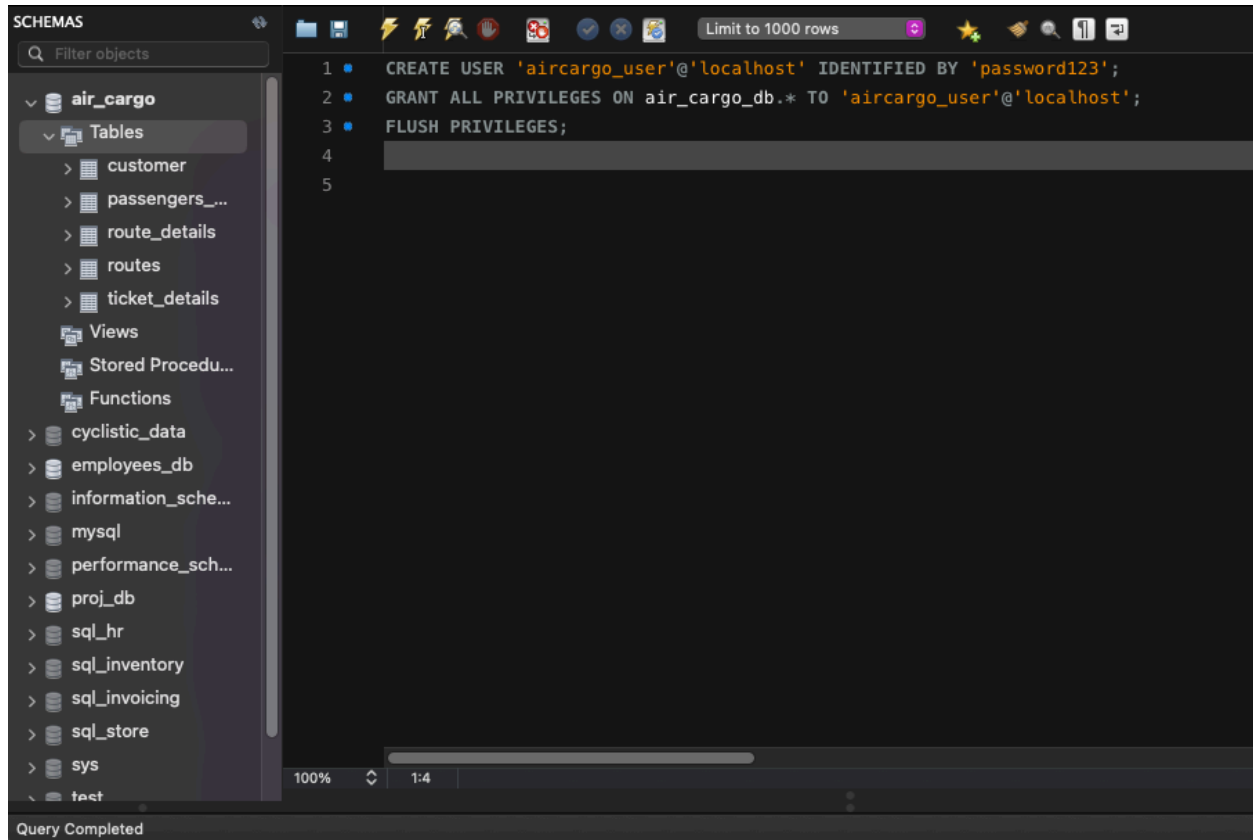
100% 1:6

Result Grid Filter Rows: Search Export:

	revenue_status
	Revenue Crossed 10000

10. Query to create and grant access to a new user to perform operations on a database.

```
CREATE USER 'aircargo_user'@'localhost' IDENTIFIED BY 'password123';  
GRANT ALL PRIVILEGES ON air_cargo_db.* TO 'aircargo_user'@'localhost';  
FLUSH PRIVILEGES;
```



11. Query to find the maximum ticket price for each class using window functions on the ticket_details table.

```
SELECT class_id,  
       price_per_ticket,  
       MAX(price_per_ticket) OVER (PARTITION BY class_id) AS max_ticket_price  
FROM ticket_details;
```

```

1 SELECT class_id,
2         price_per_ticket,
3         MAX(price_per_ticket) OVER (PARTITION BY class_id) AS max_ticket_price
4 FROM ticket_details;
5

```

100% 1:5

Result Grid Filter Rows: Search Export:

	class_id	price_per_ticket	max_ticket_price	
	Bussiness	465	510	
	Bussiness	430	510	
	Bussiness	490	510	
	Bussiness	490	510	
	Bussiness	430	510	
	Bussiness	430	510	
	Bussiness	430	510	
	Bussiness	510	510	
	Bussiness	505	510	
	Bussiness	465	510	
	Bussiness	410	510	
	Bussiness	480	510	
	Bussiness	499	510	
	Economy	120	190	
	Economy	130	190	

12. Query to extract the passengers whose route ID is 4 by improving the speed and performance of the passengers_on_flights table.

-- First, create an index on route_id for performance:

```
CREATE INDEX idx_route_id ON passengers_on_flights(route_id);
```

-- Then run the optimized query:

```
SELECT *
FROM passengers_on_flights
WHERE route_id = 4;
```

```
2 CREATE INDEX idx_route_id ON passengers_on_flights(route_id);
3
4 -- Then run the optimized query:
5 SELECT *
6 FROM passengers_on_flights
7 WHERE route_id = 4;
8
```

100% 1:8

Result Grid

	customer_id	aircraft_id	route_id	depart	arrival	seat_num	class_id	travel_date	flight_nu...
2	767-301ER	4	JFK	LAX	01E	Economy	02-09-2018	1114	
4	767-301ER	4	JFK	LAX	03FC	First Class	30-04-2020	1114	
11	767-301ER	4	JFK	LAX	05B	Bussiness	09-11-2020	1114	

passengers_on_flights 35

13. For route ID 4, write a query to view the execution plan of the passengers_on_flights table.

```
-- EXPLAIN shows the execution plan (index usage, table scans) to verify performance
improvements from indexing
EXPLAIN SELECT *
FROM passengers_on_flights
WHERE route_id = 4;
```

```

1  -- EXPLAIN shows the execution plan (index usage, table scans) to verify performance
2  EXPLAIN SELECT *
3  FROM passengers_on_flights
4  WHERE route_id = 4;
5
6

```

100% 20:4

Result Grid Filter Rows: Search Export:

	id	select_ty...	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
	1	SIMPLE	passengers_on_flights	HULL	ref	idx_route_id	idx_route_id	5	const	3	100.00	HULL

14. Query to calculate the total price of all tickets booked by a customer across different aircraft IDs using rollup function.

```

SELECT customer_id,
       aircraft_id,
       SUM(no_of_tickets * price_per_ticket) AS total_price
FROM ticket_details
GROUP BY customer_id, aircraft_id WITH ROLLUP;
# rollup --> adds subtotals per customer and a grand total for all

```

```

1 SELECT customer_id,
2         aircraft_id,
3         SUM(no_of_tickets * price_per_ticket) AS total_price
4 FROM ticket_details
5 GROUP BY customer_id, aircraft_id WITH ROLLUP;
6 # rollup --> adds subtotals per customer and a grand total for all
7
8

```

100% 1:8

Result Grid Filter Rows: Search Export:

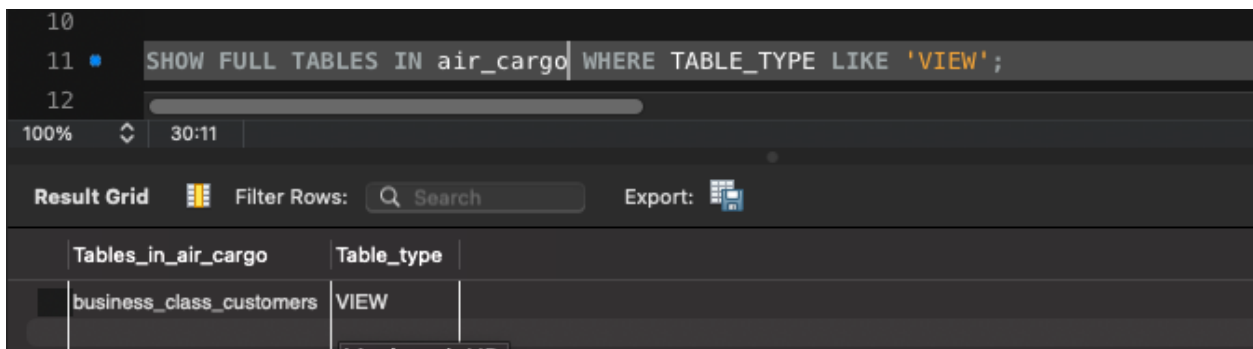
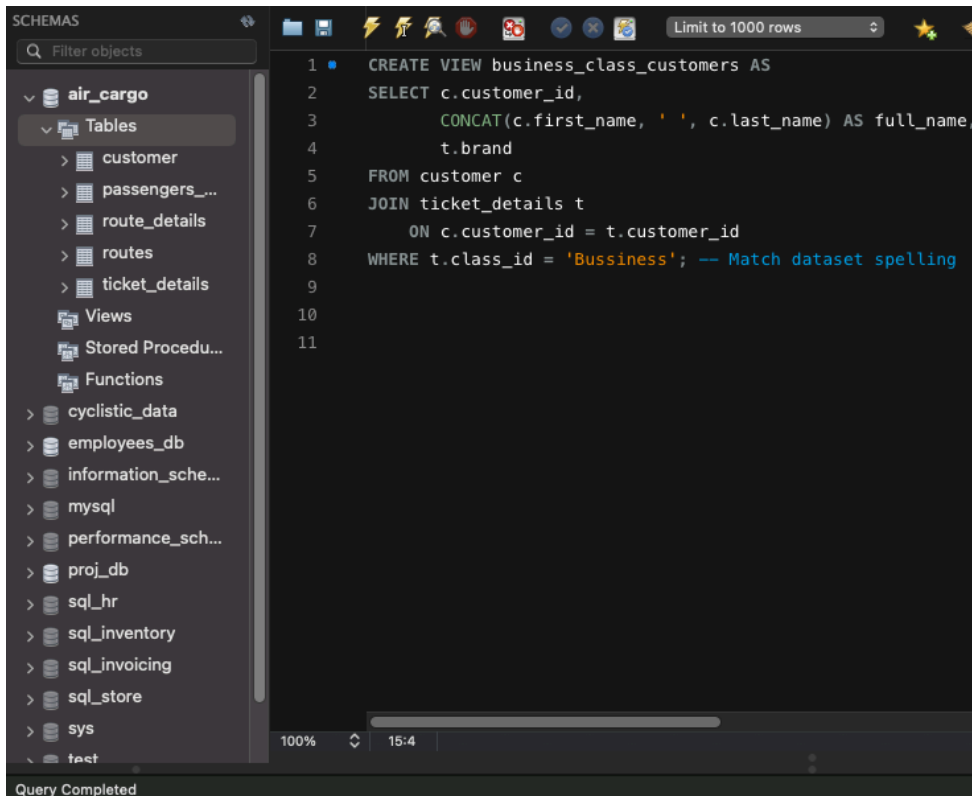
	customer_id	aircraft_id	total_pri...
1	1	CRJ900	320
1	1	ERJ142	250
1	1	NULL	570
2	2	767-301ER	130
2	2	A321	505
2	2	NULL	635
4	4	767-301ER	780
4	4	NULL	780
5	5	767-301ER	430
5	5	ERJ142	240
5	5	NULL	670
7	7	767-301ER	430
7	7	NULL	430
8	8	A321	465
8	8	NULL	465

15. Query to create a view with only business class customers along with the brand of airlines.

```

CREATE VIEW business_class_customers AS
SELECT c.customer_id,
       CONCAT(c.first_name, ' ', c.last_name) AS full_name,
       t.brand
FROM customer c
JOIN ticket_details t
  ON c.customer_id = t.customer_id
WHERE t.class_id = 'Business'; -- Match dataset spelling

```



16. Query to create a stored procedure to get the details of all passengers flying between a range of routes defined in run time and return an error message if the table doesn't exist.

```

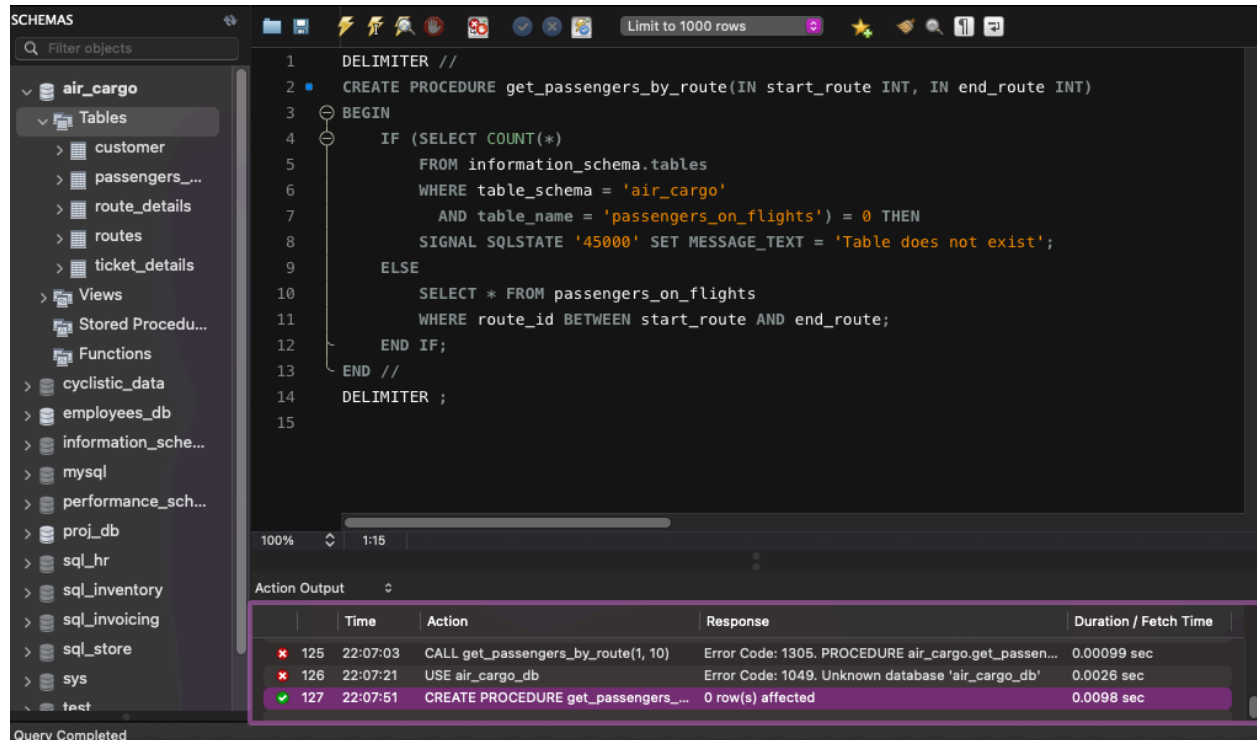
DELIMITER //
CREATE PROCEDURE get_passengers_by_route(IN start_route INT, IN end_route INT)
BEGIN
    IF (SELECT COUNT(*)
        FROM information_schema.tables
        WHERE table_schema = 'air_cargo'
        AND table_name = 'passengers_on_flights') = 0 THEN

```

```

        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Table does not exist';
    ELSE
        SELECT * FROM passengers_on_flights
        WHERE route_id BETWEEN start_route AND end_route;
    END IF;
END //
DELIMITER ;

```



- When creating a stored procedure in MySQL, it only defines and stores the procedure in the database; it does not run automatically or produce any output until you explicitly call it later using the CALL command.
 - Example use of it
 - CALL get_passengers_by_route(1, 10);

1 • CALL get_passengers_by_route(1, 10);

2

100% 1:1

Result Grid Filter Rows: Search Export:

	customer_id	aircraft_id	route_id	depart	arrival	seat_num	class_id	travel_date	flight_nu...
18		767-301ER	1	EWK	HNL	13FC	First Class	01-04-2018	1111
2		767-301ER	4	JFK	LAX	01E	Economy	02-09-2018	1114
4		767-301ER	4	JFK	LAX	03FC	First Class	30-04-2020	1114
11		767-301ER	4	JFK	LAX	05B	Bussiness	09-11-2020	1114
4		767-301ER	5	LAX	JFX	02FC	First Class	06-04-2020	1115
11		767-301ER	5	LAX	JFX	04B	Bussiness	12-11-2020	1115
46		A321	8	ORD	EWK	12FC	First Class	08-07-2011	1118
1		ERJ142	9	DEN	LAX	01EP	Economy Plus	26-12-2019	1119
29		ERJ142	9	DEN	LAX	11B	Bussiness	03-05-2018	1119
10		A321	10	HNL	DEN	05E	Economy	11-10-2020	1120

Result 39 Read On

Action Output

	Time	Action	Response	Duration / Fetch Time
✖ 126	22:07:21	USE air_cargo_db	Error Code: 1049. Unknown database 'air_cargo_db'	0.0026 sec
✔ 127	22:07:51	CREATE PROCEDURE get_passengers_...	0 row(s) affected	0.0098 sec
✔ 128	22:09:54	CALL get_passengers_by_route(1, 10)	10 row(s) returned	0.023 sec / 0.00033...

17. Query to create a stored procedure that extracts all the details from the routes table where the travelled distance is more than 2000 miles.

```

DELIMITER //
CREATE PROCEDURE get_long_routes()
BEGIN
    SELECT *
    FROM routes
    WHERE distance_miles > 2000;
END //
DELIMITER ;

call get_long_routes();

```

The screenshot shows a database IDE with the following components:

- SQL Editor:** Contains the following code:


```

1 DELIMITER //
2 CREATE PROCEDURE get_long_routes()
3 BEGIN
4     SELECT *
5     FROM routes
6 
```
- Result Grid:** Displays the results of the query. It shows a table with 7 columns: route_id, flight_nu..., origin_airp..., destination_airp..., aircraft_id, distance_mil..., and an empty column. The data is as follows:

route_id	flight_nu...	origin_airp...	destination_airp...	aircraft_id	distance_mil...	
1	1111	EWB	HNL	767-301ER	4962	
2	1112	HNL	EWB	767-301ER	4962	
3	1113	EWB	LHR	A321	3466	
4	1114	JFK	LAX	767-301ER	2475	
5	1115	LAX	JFK	767-301ER	2475	
6	1116	HNL	LAX	767-301ER	2556	
10	1120	HNL	DEN	A321	3365	
11	1121	ADL	ADL	767-301ER	4962	
- Action Output:** Shows the execution log:

	Time	Action	Response	Duration / Fetch Time
1	22:14:17	CREATE PROCEDURE get_long_routes() B...	Error Code: 1304. PROCEDURE get_long_routes alrea...	0.0018 sec
2	22:14:17	call get_long_routes()	24 row(s) returned	0.0012 sec / 0.00001.

^ 1 is an 'error,' because I already ran the query for the procedure

18. Query to create a stored procedure that groups the distance travelled by each flight into three categories. The categories are, short distance travel (SDT) for ≥ 0 AND ≤ 2000 miles, intermediate distance travel (IDT) for >2000 AND ≤ 6500 , and long-distance travel (LDT) for >6500 .

```

DELIMITER //
CREATE PROCEDURE categorize_routes()
BEGIN
    SELECT route_id, distance_miles,
           CASE
               WHEN distance_miles BETWEEN 0 AND 2000 THEN 'SDT'
               WHEN distance_miles BETWEEN 2001 AND 6500 THEN 'IDT'
               ELSE 'LDT'
           END AS distance_category
    FROM routes;
END //
DELIMITER ;

```

call categorize_routes();

The screenshot shows a SQL IDE interface. The top pane displays the SQL code for creating a stored procedure named `categorize_routes()`. The code uses a `CASE` statement to categorize routes based on distance in miles: 'SDT' for 0-2000 miles, 'IDT' for 2001-6500 miles, and 'LDT' for distances greater than 6500 miles. The bottom pane shows the 'Result Grid' with 42 rows of data. The columns are `route_id`, `distance_miles`, and `distance_category`. Below the result grid, the 'Action Output' pane shows a log of database actions, including the successful execution of the `categorize_routes()` procedure, which returned 49 rows.

```
1 CREATE PROCEDURE categorize_routes()
2 BEGIN
3     SELECT route_id, distance_miles,
4           CASE
5             WHEN distance_miles BETWEEN 0 AND 2000 THEN 'SDT'
6             WHEN distance_miles BETWEEN 2001 AND 6500 THEN 'IDT'
7             ELSE 'LDT'
8           END AS distance_category
9     FROM routes;
```

route_id	distance_miles	distance_category
1	4962	IDT
2	4962	IDT
3	3466	IDT
4	2475	IDT
5	2475	IDT
6	2556	IDT
7	1745	SDT
8	719	SDT

	Time	Action	Response	Duration / Fetch Time
✖ 1	22:14:17	CREATE PROCEDURE get_long_routes()...	Error Code: 1304. PROCEDURE get_long_routes alrea...	0.0018 sec
✔ 2	22:14:17	call get_long_routes()	24 row(s) returned	0.0012 sec / 0.00001...
✔ 3	22:15:34	CREATE PROCEDURE categorize_routes...	0 row(s) affected	0.0078 sec
✔ 4	22:15:34	call categorize_routes()	49 row(s) returned	0.0015 sec / 0.00001...

- CASE statement to categorize routes:
 - SDT: Short Distance Travel (0–2000 miles)
 - IDT: Intermediate Distance Travel (2001–6500 miles)
 - LDT: Long Distance Travel (>6500 miles)

19. Query to extract ticket purchase date, customer ID, class ID and specify if the complimentary services are provided for the specific class using a stored function in stored procedure on the ticket_details table.

a. Condition:

- If the class is Business and Economy Plus, then complimentary services are given as Yes, else it is No**

DELIMITER //

-- Step 1: Create the stored function

CREATE FUNCTION complimentary_services(class_name VARCHAR(50))

```

RETURNS VARCHAR(3)
DETERMINISTIC
BEGIN
    -- If class is Bussiness or Economy Plus, return 'Yes'; otherwise return 'No'
    RETURN IF(class_name IN ('Bussiness', 'Economy Plus'), 'Yes', 'No');
END //

-- Step 2: Create the stored procedure that calls the function
CREATE PROCEDURE get_ticket_services()
BEGIN
    SELECT
        p_date,
        customer_id,
        class_id,
        complimentary_services(class_id) AS complimentary -- Calls the function
    FROM ticket_details;
END //

DELIMITER ;

```

6 DETERMINISTIC

7 BEGIN

8 -- If class is Bussiness or Economy Plus, return 'Yes'; otherwise return 'No'

9 RETURN IF(class_name IN ('Bussiness', 'Economy Plus'), 'Yes', 'No');

10 END //

11

12 -- Step 2: Create the stored procedure that calls the function

13 CREATE PROCEDURE get_ticket_services()

14 BEGIN

15 SELECT

16 p_date,

17 customer_id,

18 class_id,

19 complimentary_services(class_id) AS complimentary -- Calls the function

20 FROM ticket_details;

21 END //

22

23 DELIMITER ;

100% 12:23

Action Output

	Time	Action	Response	Duration / Fetch Time
✓ 12	22:24:09	USE air_cargo	0 row(s) affected	0.00067 sec
✓ 13	22:25:01	-- Step 1: Create the stored function CR...	0 row(s) affected	0.0063 sec
✓ 14	22:25:01	CREATE PROCEDURE get_ticket_servic...	0 row(s) affected	0.0010 sec

- I am calling for my satisfaction
 -- Step 3: Call the procedure to run it
 CALL get_ticket_services();

The screenshot shows a SQL IDE interface. At the top, a query is entered in a text editor:

```
1  -- Step 3: Call the procedure to run it
2  CALL get_ticket_services();
3
```

Below the editor, the 'Result Grid' is displayed, showing a table with 5 columns: p_date, customer_id, class_id, compliment..., and an empty column. The table contains 9 rows of data:

p_date	customer_id	class_id	complimenta...	
26-12-2018	27	Economy	No	
02-02-2020	22	Economy Plus	Yes	
03-03-2020	21	Bussiness	Yes	
04-04-2020	4	First Class	No	
05-05-2020	5	Economy	No	
07-07-2020	7	Bussiness	Yes	
08-08-2020	8	Economy Plus	Yes	
09-09-2020	9	First Class	No	

Below the result grid, the 'Action Output' section shows a log of executed actions:

	Time	Action	Response	Duration / Fetch Time
✓ 13	22:25:01	-- Step 1: Create the stored function CR...	0 row(s) affected	0.0063 sec
✓ 14	22:25:01	CREATE PROCEDURE get_ticket_servic...	0 row(s) affected	0.0010 sec
✓ 15	22:25:35	CALL get_ticket_services()	50 row(s) returned	0.0069 sec / 0.0000...

20. Query to extract the first record of the customer whose last name ends with Scott using a cursor from the customer table.

```
DELIMITER //
```

```
CREATE PROCEDURE get_scott_customer()
```

```
BEGIN
```

```
    DECLARE done INT DEFAULT 0; -- Flag to detect end of cursor
```

```
    DECLARE cust_id INT;
```

```
    DECLARE cust_name VARCHAR(100);
```

```
    -- Cursor to fetch customers whose last name ends with 'Scott'
```

```
    DECLARE cur CURSOR FOR
```

```
        SELECT customer_id, CONCAT(first_name, ' ', last_name)
```

```
        FROM customer
```

```
        WHERE last_name LIKE '%Scott';
```

```
-- Handler to set done flag when no more rows are found, will through error if not used  
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
```

```
OPEN cur; -- Open cursor  
FETCH cur INTO cust_id, cust_name; -- Fetch first record  
SELECT cust_id AS customer_id, cust_name AS full_name; -- Return result  
CLOSE cur; -- Close cursor  
END //
```

```
DELIMITER ;
```

```
FORGOT TO SCREENSHOT: CREATE PROCEDURE get_scott_customer() BEGIN  
DECLARE done INT DEFAULT 0; -- Flag to detect end of cursor  DECLARE cust_id  
INT;  DECLARE cust_name VARCHAR(100);  -- Cursor to fetch customers whose last  
name ends with 'Scott'  DECLARE cur CURSOR FOR      SELECT customer_id,  
CONCAT(first_name, ' ', last_name)      FROM customer      WHERE last_name LIKE  
'%Scott';  -- Handler to set done flag when no more rows are found  DECLARE  
CONTINUE HANDLER FOR NOT FOUND SET done = 1;  OPEN cur; -- Open cursor  
FETCH cur INTO cust_id, cust_name; -- Fetch first record  SELECT cust_id AS  
customer_id, cust_name AS full_name; -- Return result  CLOSE cur; -- Close cursor  
END
```

```
-- Call the procedure  
CALL get_scott_customer();
```

```
1  -- Call the procedure
2  CALL get_scott_customer();
3
```

100% 1:3

Result Grid Filter Rows: Search Export:

customer_id	full_name
37	Samuel Scott

Result 3 Read

Action Output

	Time	Action	Response	Duration / Fetch Time
✓ 15	22:25:35	CALL get_ticket_services()	50 row(s) returned	0.0069 sec / 0.00002
✓ 16	22:26:56	CREATE PROCEDURE get_scott_custom...	0 row(s) affected	0.0068 sec
✓ 17	22:27:55	CALL get_scott_customer()	1 row(s) returned	0.0081 sec / 0.00001

Stored Procedu...

- categorize_r...
- get_long_rou...
- get_passeng...